

AerX Threat-Aware Route Engine — Project Handbook

Purpose of this document: a complete, self-contained reference so that **you** (or a future AI session opening this repo cold) can understand every part of the project and continue building on it. It covers what the project is, how every module works, the algorithms and math, the exact tuning constants and **why** they are what they are, the design decisions, the debugging history, known limits, future work, and how to run and extend everything.

- Repo: <https://github.com/harshwardhangiri/aerx-tactical-route-planner>
 - Author: Harshwardhan Giri Goswami (IIT Kanpur, Aerospace Engineering)
 - Language/stack: Python 3.13 · NumPy · SciPy · rasterio · matplotlib · Plotly · CesiumJS
 - Entry point: `main.py` (one CLI). Interactive tool: `interactive.py`. Viewer server: `serve_viewer.py`.
 - Tests: `python -m unittest discover tests` → 15 tests, all passing.
-

1. What the project is (one paragraph)

A terrain-aware, risk-sensitive 3D UAV/helicopter mission-planning simulator. A conventional planner finds the **shortest** path; this engine finds the most survivable full sortie — fly in (ingress), survive radar/SAM exposure by hiding behind real terrain, and get home (egress) on one fuel budget — over real Copernicus GLO-30 elevation data for anywhere on Earth. It scores every candidate route on survivability, fuel, clearance and efficiency, and visualizes it as a dashboard, a Plotly 3D view, and a CesiumJS georeferenced globe flythrough. It is calibrated to real aircraft (speed/fuel/turn/climb/RCS) and reports real km, minutes, kg of fuel and kinematic feasibility.

Safety framing (keep this honest everywhere): all hazards are generic simulated sensors with normalized, illustrative parameters — no real operational RCS/weapon/radar values. It is a **research-level simulation**, not operationally deployable.

2. Project history / evolution (so you know how it got here)

Build order over the sessions (each is a git commit or phase):

1. Initial engine (`ccc8b39`) — 6 scenarios (4 synthetic + 2 real DEM), D* Lite, RRT* refiner, radar model, ingress/egress, fuel budget, mission score, Monte Carlo, matplotlib/Plotly/Cesium viz. Normalized units, no specific aircraft.
2. Performance — the risk-aware smoothing's live line-of-sight risk was the bottleneck; replaced with a cached trilinear grid lookup (`grid.risk_at`). A full solve dropped ~110 s → ~30 s.
3. Consolidation — removed the 4 synthetic scenarios and the `run_analysis.py` / `run_region.py` scripts; folded everything into a single `main.py` CLI (`everest` / `ghats` / `region`). See memory `aerx-single-entry-point`.

4. Terrain-driven threats — replaced fixed radar coordinates with `auto_place_hazards()` that places radars on the region's own summits/ridges.
 5. Survivability saturation fix (`fd3de8d`) — high-relief regions read 0% survivability everywhere (no "best route"). Root cause + fix in §7 and §11.
 6. README gallery + Cesium flythrough GIF (`cf09f98`, `a2518ad`).
 7. Upgrades (`6cb312e`, `b782d3f`, `90c4b6f`) — real aircraft profiles, interactive radar placement, standalone RRT* planner, RRT* refinement made the default, and a kinematic-feasibility readout.
-

3. Architecture — the pipeline

One vertical slice: data → algorithm → score → visualization.

1. Terrain / DEM (`terrain.py`, `los.py`) — load a real Copernicus GLO-30 tile via rasterio; height anywhere by bilinear interpolation.
 2. Line-of-sight masking (`los.py`) — sample the radar→aircraft ray; terrain above the sightline masks detection (this is why hiding in valleys works).
 3. Threat field (`risk_field.py`) — each radar/SAM is a 3D detection volume; detectability follows the radar equation, gated by range, LOS, sector, cone, min-altitude, and combined across emitters by probabilistic union.
 4. Planning grid + cost (`planning_grid.py`) — airspace → (x,y,altitude) nodes; below-terrain / low-clearance nodes forbidden; each edge costs distance + risk + climb-energy + clearance.
 5. Global planner (`dstar_lite.py` or `rrt_star.py`) — D* Lite (default) or a from-scratch RRT*; plans ingress (base→target) and egress (target→base).
 6. Local refinement (`rrt_star.py`, `smoothing.py`) — aircraft-aware RRT* refinement (default) → flyable trajectory, then B-spline smoothing.
 7. Metrics (`route_metrics.py`) — distance, time, energy, integrated & peak risk, min AGL, climb angles, per leg.
 8. Scoring (`scoring.py`) — round-trip survivability, fuel-budget check, mission score /100.
 9. Real-world calibration (`aircraft.py`) — convert to real km/min/kg + turn/bank kinematic feasibility for the selected airframe.
 10. Monte Carlo (`monte_carlo.py`) — re-score under jitter → a distribution.
 11. Visualization (`visualization.py`, `web_export.py`, `cesium_export.py`).
-

4. File-by-file reference (src/ + root)

- `terrain.py` — Copernicus GLO-30 loader. `download_copernicus_tile(lat,lon)` fetches the tile from the public AWS S3 bucket (no auth) into `data/terrain/`; `copernicus_tile_name(lat,lon)`; `load_real_dem_region(path,row0,col0,win_px,out_cells)` crops+downsamples; `find_relief_window(path)` auto-picks a navigable-relief crop. `create_mountain_pass_terrain` / `create_demo_terrain` are test fixtures only (a twin-peak synthetic terrain for the LOS-masking unit test).
- `los.py` — `terrain_height(terrain,x,y)` bilinear interpolation (terrain is indexed `[y,x]`); `check_los(terrain, a, b, samples)` ray-marches the sightline.
- `risk_field.py` — the threat model. `HazardSite` dataclass (3D detection

volume: position, radius=R_max, intensity, requires_los, decay_type, threat_type
radar/sam/aaa, min_detect_alt, azimuth sector, elevation cone, `covers()`).
`compute_hazard_attenuation()` (radar equation), `compute_point_risk()` (all
gates + union), `compute_risk_slice()` (2D heatmap). `auto_place_hazards()` places
threats from terrain+corridor. `load_mission_scenario('dem' | 'ghats' | 'custom')` +
`_finalize_real_dem()` build a scenario. `get_last_region_scale()` returns the
real m/cell + relief for the calibration layer. `set_custom_dem()` registers a
region for the `custom` scenario.

- `planning_grid.py` — `PlanningGrid3D` (nodes, `xy_scale` m/cell, `z_layers` ASL, min/max AGL, climb-slope limit, precomputed risk cache with `risk_at()` trilinear lookup, `add_hazard()` for incremental replan). `CostWeights` (distance/risk/energy/altitude; `.distance_focused()`, `.risk_focused()`, `.balanced()`).

- `dstar_lite.py` — `DStarLite3D`: incremental optimal graph search (backward from goal; g/rhs values; priority queue; `plan()`, `replan_after_hazard_change()`, `last_expansions`).

- `rrt_star.py` — `RiskAwareRRTStar` (proper RRT*: nearest-neighbor, steering, rewiring, informed tube sampling OR uniform global sampling, kinematic limits, risk-aware shortcutting). `refine_with_rrt_star()` (refiner) and `plan_global_rrt_star()` (from-scratch planner).

- `smoothing.py` — `smooth_trajectory()`: risk-aware LOS waypoint pruning + B-spline (scipy `splprep/splev`), with a risk guard so shortcuts don't cut through avoided threats; takes `risk_sampler=grid.risk_at` for speed.

- `route_metrics.py` — `compute_route_metrics()` → `RouteMetrics` (distance, time, energy, integrated/peak risk, min/max/mean AGL, climb/descent angles).

- `scoring.py` — `evaluate_round_trip()`, `apply_fuel_budget()`, `compute_mission_score()` → `MissionScore`; survivability $S = \exp(-\lambda R_{int})$.

- `aircraft.py` — `RegionScale`, `Aircraft` registry, `realize_sortie()` (real km/min/kg + turn-radius/bank kinematic feasibility), `kinematic_limits_for_grid()` (aircraft turn radius + climb → normalized RRT* limits).

- `monte_carlo.py` — `run_sortie_monte_carlo()` perturbs both legs + shared threats, holds fuel fixed via `nominal_fuel_factor`, returns P5–P95 distribution.

- `analysis.py` — `pareto_sweep()` (re-plan across risk weights), `replanning_demo()` (pop-up threat repair), and their render functions.

- `visualization.py` — `render_tactical_dashboard()` (4-panel), `render_mission_scorecard()`, `InteractiveMissionDashboard` (the renderer class; its live scenario-switcher is vestigial now).

- `web_export.py` — Plotly 3D HTML. `cesium_export.py` — CesiumJS globe HTML (jsDelivr CDN, local ENU frame, point-cloud terrain, route polylines, capped threat domes, flythrough timeline; left-drag orbits).

- `render_env.py` — matplotlib backend selection (Agg headless / TkAgg interactive).
- `main.py` — the CLI + `solve_mission_scenario()` + `run_full_suite()` + region download.
- `interactive.py` — click-to-place radar planner (matplotlib GUI).
- `serve_viewer.py` — local http server that opens the newest `*_cesium.html`.

5. Algorithms in depth

D* Lite (global, default). Incremental optimal graph search; like A* but keeps its search so a changed edge cost (pop-up threat) is repaired by touching only affected nodes ($\sim 3\times$ cheaper than re-planning). Searches backward from the goal so cost-to-goal values survive when the start moves. Uses g and rhs (one-step lookahead); a node is consistent when $g=rhs$; a priority queue holds inconsistent nodes, keyed by $\min(g,rhs)+h$ where the heuristic h estimates distance to the START (because the search targets the start). Admissible heuristic $\Rightarrow$ optimal.

RRT* (local refiner AND standalone planner). A proper RRT*: sample $\rightarrow$ nearest neighbor $\rightarrow$ steer (step_size) $\rightarrow$ choose best kinematically-valid parent within a capped neighborhood $\rightarrow$ rewire neighbors $\rightarrow$ connect to goal $\rightarrow$ randomized risk-aware shortcutting. Two modes: refiner (informed tube sampling around the D* guide, `refine_with_rrt_star`) and global planner (uniform airspace sampling, `plan_global_rrt_star`, `--planner rrt`). Kinematic limits (max heading change per edge, max climb gradient) come from the aircraft's real turn radius + climb rate.

Radar detection. Detectability follows the radar equation (inverse 4th power of range — two-way spreading), gated by range (0 beyond R_{max}), line-of-sight (0 if terrain blocks), and the 3D volume (sector/cone/min-altitude). Combined across emitters by probabilistic union. LOS is a binary multiplier on the radar value.

Survivability. $S = \exp(-\lambda \cdot R_{int})$ — surviving the whole flight means surviving every segment, and independent survivals multiply $\rightarrow$ an exponential of integrated exposure (Poisson zero-detection). Round trip sums both legs.

Monte Carlo. Perturb waypoints (tracking error) + threat positions/intensity (intel uncertainty) $\sim 28-50\times$, re-score $\rightarrow$ mean/std/P5/P50/P95 (fuel held fixed).

Pareto sweep. Re-plan across risk weights; plot distance vs exposure $\rightarrow$ the trade-off front; the "knee" justifies the balanced weight. It *explains* the weight; the mission score selects the route. Chain: `D\* finds route for a cost  $\rightarrow$  Pareto justifies the weight  $\rightarrow$  mission score picks the winner`.

Auto threat placement. Reads the terrain + base $\rightarrow$ target corridor; a **Search Radar** on the tallest summit near the corridor, a **Sector Radar** on a flanking ridge, a SAM on high ground over the corridor midpoint (kept clear of the radars). Deterministic (no randomness). See §7 for the radius tuning.

6. The math (formulas + where they come from)

- Radar detectability: $M(r) = intensity \cdot 0.05 \cdot (R_{max} / r)^4$, range- and LOS-gated. (Two-way spreading $\rightarrow$ 4th power. From Drones 2026, 10, 469, Eq. 27.)

- Multi-radar union: $R(x,y,z) = 1 - \prod_i (1 - R_i)$ (stays in $[0,1]$).

- Integrated exposure: $R_{int} = \int R \, d\tau$ (trapezoidal along the flown path).

- Survivability: $S = \exp(-\lambda \cdot R_{int})$, $\lambda = 0.05$. Round trip:

$S = \exp(-\lambda \cdot (R_{ingress} + R_{egress}))$. (Poisson survival; matches the project's

own Stage-1 concept $P_{survive} = \exp(-\int \lambda_{detect} \, dt)$; textbook aircraft-survivability form, Ball.)

- Mission score: $100 \cdot (0.5 \cdot S + 0.25 \cdot dist + 0.15 \cdot time + 0.10 \cdot clear) \cdot fuel_factor$,

`fuel_factor = 0.5` if over the fuel budget.

- Fuel budget: $1.15 \times \text{energy of the most efficient candidate sortie}$.
- Turn radius (coordinated turn): $R = v^2 / (g \cdot \tan(\text{bank}))$.
- Required bank of a route turn: $\text{bank} = \text{atan}(v^2 / (g \cdot R_{\text{local}}))$, with `R_local` from the Menger curvature of consecutive waypoints (metres).
- LOS test: sample the radar→point ray; blocked if terrain height > sightline anywhere. Bilinear/trilinear interpolation for terrain height and risk lookup.

7. Key tuning constants & WHY (the hard-won knowledge)

- `Z_RISK_SCALE = 0.35` (`risk_field.py`) — the DEM is normalized to a fixed band [90,350], which exaggerates vertical relief ~2× vs horizontal. The detection-range slant is 3D, so uncorrected a summit radar is all-or-nothing (whole-map coverage → every route 0% survivable). De-weighting the vertical in the *range* gate gives each radar a real horizontal footprint. (Angular cone gate still uses true geometry.)

A unit test derives its expected value from this constant.

- Auto-place radar radius: `base_R = 0.60 * span`, then calibrated per region: scale R down until the straight base→target corridor's integrated exposure ≤ `TARGET = 56.0`. This equalizes "direct path threatened, survivable detour exists" across regions so survivability never collapses to all-0%.

- `RADAR_BOUNDARY_DETECTABILITY = 0.05` — detectability at `R_max` (calibration eta).
- `THREAT_TYPE_WEIGHT` — radar 1.0, sam 1.3, aaa 0.85 (relative lethality).
- `RCS_FRONT=0.6, RCS_SIDE=1.5` — optional anisotropic RCS (default off).
- $\lambda = 0.05$ (`risk_sensitivity`) — survivability sensitivity; tuned so the spread is meaningful across regions.

- Mission weights: survive 0.50, distance 0.25, time 0.15, clearance 0.10.
- Fuel reserve: 1.15×.
- RRT* config defaults (`RRTStarConfig`): `max_iterations` 1500 (refine uses 800, global uses 6000), `step_size` 6.0, `neighbor_radius` 14.0, `min_agl` 15, `risk_weight` 40 (refiner uses `weights.w_risk*6`; global uses `w_risk*8` so the distance-focused profile isn't silently risk-avoided), `max_turn_deg` 50, `max_climb_rate` 0.7 (both overridden per-aircraft via `kinematic_limits_for_grid`).
- Grid params (`main.py`): relief>400 m → `min_agl` 25, `max_agl` relief+120, `climb` 1.0; else 15, 220, 0.85 (adaptive to terrain relief).
- Real scale: `meters_per_cell = 150` (Copernicus 30 m/px × 600 px ÷ 120 out-cells); `meters_per_zunit = real_relief / 260`.

8. Real-world calibration layer (`aircraft.py`)

Design choice: a calibration layer on top of the tuned normalized planner, NOT a core-units rewrite (to avoid breaking the survivability model / `Z_RISK_SCALE` / radius calibration). The planner still runs in the grid; outputs and kinematic limits are calibrated to real scale + aircraft.

- `RegionScale` — `meters_per_cell` (150), `relief_m`, `elev_min_m`; `real_alt_m(z)`. Captured in `_finalize_real_dem` via `get_last_region_scale()`.

- **Aircraft** registry (edit here to add platforms): `scout_heli` (default), `chinook`, `quad_uav`, `reaper`. Fields: `cruise/max speed (m/s)`, `climb_rate (m/s)`, `max_bank_deg`, `min_agl`, `service_ceiling`, `fuel_capacity_kg`, `cruise_burn_kgps`, `climb_burn_factor`, `rcs_m2`. Methods: `turn_radius()`, `endurance_s()`, `range_m()`.
 - `realize_sortie(legs, terrain, scale, aircraft) → RealStats`: `distance_km`, `time_min`, `fuel_kg/%`, `within_endurance`, `min_agl_m`, `max_alt_m`, `within_ceiling`, `min_route_radius_m` (tightest turn via Menger curvature), `max_bank_deg` (required bank), `within_bank`, `within_climb`, `flyable (= within_bank; turn is the hard limit, steep climb is soft since a heli slows to climb)`.
 - `kinematic_limits_for_grid(aircraft, scale, step_cells) → (max_turn_deg, max_climb_rate)` for the RRT* in normalized units.
-

9. Design decisions & rationale

- D* Lite over A*/RTA* — incremental cheap repair for pop-up threats; optimal global route, not a per-step reflex. Kept as the default global planner.
 - Global planner + local kinematic refinement — D* Lite finds **where** to go (optimal, repairable); aircraft-aware RRT* makes it **flyable** (turn/climb). This is the standard autonomy architecture; RRT* refinement is now the default.
 - Real DEMs over synthetic — generalizes to genuine terrain, any lat/lon.
 - Auto threat placement — a DEM has no radars; hand-placing is arbitrary and doesn't generalize; placement is read from terrain + corridor.
 - `exp()` survivability — survival compounds multiplicatively; stays in [0,1], unlike "1 – total risk".
 - Calibration layer, not core-units rewrite — real aircraft values without destabilizing the tuned risk model.
 - Turn radius = hard kinematic limit; steep climb = soft — a helicopter slows to climb (the time/fuel model captures it).
-

10. Known limitations & future work

- Normalized planning grid — planning runs in a scaled grid; real numbers come from the calibration layer. A full real-units core is possible but riskier.
- Everest saturates — its extreme, uniformly-high crop offers almost no masking; even long detours stay exposed (0% on all profiles). Genuine terrain property. Fix idea: per-region adaptive threat strength targeting a survivability band.
- Threat placement is a deterministic terrain heuristic, not real intelligence. Future: ingest a real order-of-battle / threat list.
- Radar model is normalized/illustrative — no operational radar/weapon values. Future: validated sensor models, clutter, jamming, multipath.
- 2.5-D terrain (heightmap) — no overhangs/buildings/vegetation.
- Static threats — no moving SAMs, sensor fusion, or weather beyond MC jitter.
- Post-RRT B-spline smoothing is not climb-constrained — can introduce a locally-steep climb (flagged, not failed, by the kinematic readout). Future: a climb-aware final smoother, or fold smoothing into the RRT*.
- No real-time control layer — plans offline; an MPC / receding-horizon

controller (possibly RL-tuned) is future work, keeping the deterministic planner as the explainable backbone.

- RRT* standalone can be slow / occasionally fall back to D* Lite on narrow corridors (probabilistically complete only in the limit).

11. Debugging history (the war stories)

- All-zero survivability ([fd3de8d](#)) — after terrain-placed threats, high-relief regions read 0% on every route (no best-route signal). Cause: the normalized DEM exaggerates vertical relief $\sim 2\times$, so the 3D detection-range slant was vertical-dominated and a summit radar covered the whole map. Fix: `Z_RISK_SCALE = 0.35` (horizontal footprint) + per-region range calibration to `TARGET = 56`. Kept 15/15 tests green.
- Performance 110 s $\rightarrow$ 30 s — risk-aware smoothing did live LOS risk per shortcut ($O(N^2)$); replaced with the grid's cached trilinear `risk_at` lookup.
- Profile-spread collapse — when RRT* refinement became default, its fixed `risk_weight=40` risk-avoided *every* profile (distance-focused jumped 34% $\rightarrow$ 64%). Fix: the refiner's risk weight now tracks each profile's `w_risk`.
- Cesium giant threat bubbles — the true radar range is $\sim$ map-sized; drawn full with 40 $\times$ vertical exaggeration it swallowed the map. Fix: cap the rendered dome to $\sim 28\%$ of the crop (the real field lives in the 2D heatmap).
- Cesium orbit — default globe-pan on left-drag felt like "nothing rotates"; remapped left-drag to orbit.

12. How to run everything

```
# tests
python -m unittest discover tests

# a mission (real DEM, full product suite -> results/<region>/<timestamp>/)
python main.py ghats
python main.py everest
python main.py region --lat 34.1 --lon 74.8 --label kashmir

# options
python main.py ghats --quick # scorecard + dashboard only (fast)
python main.py ghats --aircraft chinook # scout_heli | chinook | quad_uav | reaper
python main.py ghats --planner rrt # from-scratch RRT* global planner (else D* Lite)
python main.py ghats --refine none # B-spline only (default is aircraft-aware RRT*)

# interactive radar placement (needs a display)
python interactive.py ghats
python interactive.py region --lat 46.5 --lon 8.0

# CesiumJS viewer (serve over http; left-drag orbits)
python serve_viewer.py
```

Outputs per run: `<region>_scorecard.png`, `_dashboard.png`, `_pareto.png`, `_replanning.png`, `_3d.html` (Plotly), `_cesium.html` (CesiumJS). `--quick` stops after the scorecard + dashboard. `results/` and the DEM tiles are gitignored.

13. Repo & git

- Structure: `src/` (engine), `tests/` (15 unit tests), `assets/` (README gallery images + flythrough gif), `data/terrain/` (cached DEM tiles, gitignored), `results/` (run outputs, gitignored), `main.py`, `interactive.py`, `serve_viewer.py`, `AerX_Route_Engine_Brief.html`, `README.md`, this handbook.
 - Commit history (newest first): `90c4b6f` kinematic readout · `e2caad5` brief numbers · `b782d3f` RRT* default · `6cb312e` aircraft/interactive/RRT · `a2518ad` flythrough gif · `cf09f98` gallery · `fd3de8d` survivability fix · `ccc8b39` initial engine.
 - Gotchas: GitHub is blocked on IIT Kanpur campus WiFi (works on phone hotspot); commit messages may carry a `Co-Authored-By: Claude` trailer (strip via `git filter-branch --msg-filter "sed '/Co-Authored-By: Claude/d'"` + force-push if you don't want it); console em-dashes show as `?` on Windows cp1252 (cosmetic).
-

14. How to extend it (recipes)

- Add an aircraft: add an `Aircraft(...)` entry to the `AIRCRAFT` dict in `src/aircraft.py` (real speed/climb/bank/fuel/rcs). It auto-appears in `--aircraft`.
 - Add a built-in region: add a branch in `load_mission_scenario()` that loads a tile and returns `_finalize_real_dem(raw, title, desc)`, and a `REGIONS` entry in `main.py`. (Any lat/lon already works via `region`.)
 - Change threat placement: edit `auto_place_hazards()` in `risk_field.py` (positions) and the radius calibration (`base_R`, `TARGET`).
 - Add a planner: implement it, then branch in `main.py`'s `_plan_raw()` on a new `--planner` choice (keep a D* Lite fallback).
 - Tune survivability spread: λ (scoring), `Z_RISK_SCALE` and `TARGET` (`risk_field`). Re-run `main.py ghats --quick` and check the profile spread.
 - Put flyability on the PNG scorecard: it's console-only today; add it in `visualization.render_mission_scorecard()` using `routes_data[name]["real"]`.
-

15. Fast orientation for a future AI session

Read `MEMORY.md` in the memory dir, then this handbook, then `README.md`. The engine is `main.py` → `solve_mission_scenario()`. The risk model is `risk_field.py` (especially `compute_point_risk`, `auto_place_hazards`, `Z_RISK_SCALE`). Real-aircraft calibration is `aircraft.py`. Always run `python -m unittest discover tests` (expect 15 OK) and `python -m pyflakes main.py src/*.py` after changes. Keep the safety framing honest. The single most load-bearing non-obvious fact: `**terrain` is normalized to `[90,350]` and `Z_RISK_SCALE=0.35` de-weights the vertical so radars have a real horizontal footprint** — do not "simplify" that away.